Reproducing "Near-Duplicate Detection for Web Crawling"

Name Surname

2026-08-01

Abstract This paper aims to reproduce the findings of the "Near-Duplicate Detection for Web Crawling" work, using PySpark to scale the proposed algorithm, under different datasets, feature models and parameters.

Introduction

Detecting duplicates in a dataset is crucial for tasks like web crawlers or AI model training, freeing compute power. In order to solve this, intuitively, it's possible to hash data and compare all rows using a cartesian product. This approach is fragile - tiny changes between data will produce wildly different hashes, which is undesirable. Moreover, the time complexity for a cartesian product is $O(n^2)$, which is untractable for large datasets. Manku et al. approach [1] suggest to look for near duplicates. It aims to produce similar hashes for similar data, while providing optimization for the comparison algorithm. Their feature model proposals are benchmarked against the PAWS dataset [2], a corpus of paraphrased sentences, and a synthetic Wikipedia dataset [3]. Lastly, their algorithm is ran on a subset of the C5 dataset [4], a collection of web crawls.

Methodology

Manku et al. [1] proposal can be split into a two part pipeline.

Fingerprint Generation

The target of this first phase is to transform text into binary values of fixed length, called *fingerprints*. More specifically, to similar data must corrispond similar fingerprints (i.e. their hamming distance is low). In order to do this, Charikar's SimHash [5] is used: each text is converted into a set of fixed length binary values called *features*, which are then assembled into a single binary value of the same size - a fingerprint. Choosing the correct feature model and fingerprint size for a dataset is important. The original paper proposes different feature models like *Shingles* and *TF-IDF*, ultimately choosing the latter, with a fingerprint size of 64 bits. In this work, four feature models are tested and compared: *Shingles*, *TF-IDF*, *Bag of Words* and *Embeddings*, with fingerprint sizes from 32 bits up to 512 bits.

Shingles

…

TF-IDF

…

Bag of Words

…

Embeddings

…

Fingerprint Aggregation

…

Implementation

This implementation leverages the PySpark [6] framework, which allows to scale computing resources seamlessly.

Results

This is the results section.

Discussion

This is the discussion section.

Conclusion

This is the conclusion section.

Bibliography

[1] A. D. S. Gurmeet Singh Manku Arvind Jain, "Detecting Near-Duplicates for Web Crawling," 2007.

[2] Y. Zhang, J. Baldridge, and L. He, "PAWS: Paraphrase Adversaries from Word Scrambling," in *Proc. of NAACL*, 2019.

[3] W. Foundation, "Wikimedia Downloads." [Online]. Available: https://dumps.wikimedia.org/

[4] B. Vanroy, " CommonCrawl CreativeCommons Corpus (C5) ." [Online]. Available: https://huggingface.co/datasets/BramVanroy/CommonCrawl-CreativeCommons

[5] M. S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," 2002.

[6] [Online]. Available: https://spark.apache.org/docs/latest/api/python/index.html